COMS30020 - Computer Graphics

Week 6 Briefing

Dr Simon Lock

Mid Term Test

Thanks to everyone who attended mid-term test

Currently in the process of marking all the scripts

We should get through them sometime this week

Will release marks and sample solutions shortly

This Week's Workbook

In this week's workbook, we will be changing tack
Back from reading week, time for a new direction !

Up until now, we have approached rendering by...
Projecting 3D vertices DOWN onto 2D image plane

In future, we will switch things around completely...
Reaching OUT from camera/plane INTO 3D scene

RayTracer

Overall Approach

```
FOR each pixel on the image plane {
   Fire a ray into the 3D scene
   FOR each triangle in the model(s) {
      Check IF ray intersects with the triangle {
         Remember this triangle and distance to it
      }
   }
   Get closest triangle and paint pixel same colour
}
```

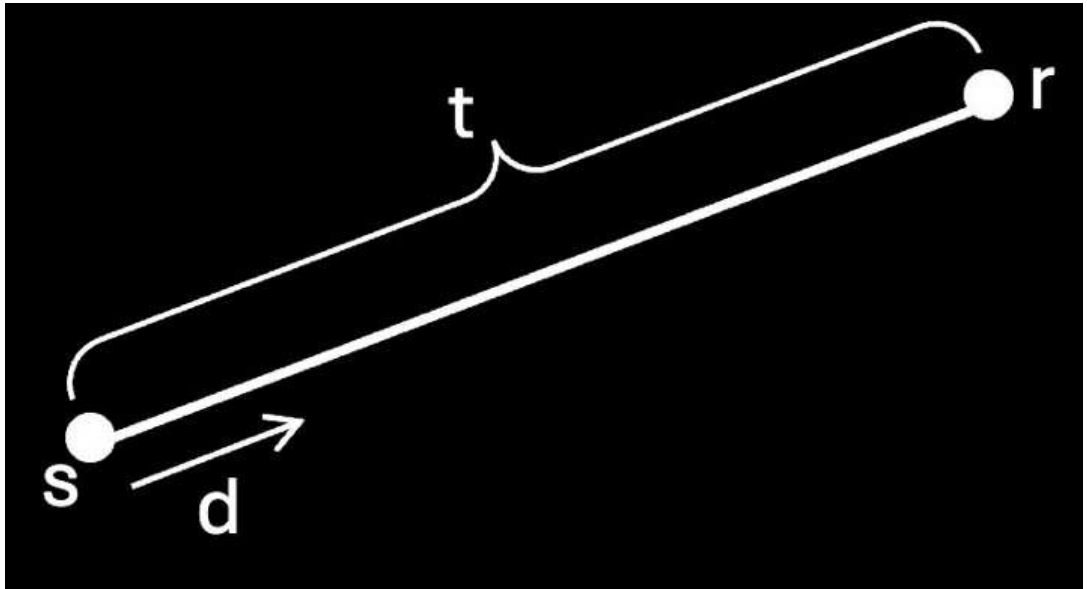
Looks like we are going to need a bit of maths !
Remember that linear algebra from "Maths for CS" ?
Maybe all of that work was worthwhile after all !!!

A Point on a Ray

A position 'r' along a ray can be represented as:

- a starting point 's' (in our case - the camera) plus...
- a distance 't' in the direction of a unitary vector 'd':

```
r = s + (t * d)
```



A Point on a Triangle

A point 'r' on a triangle can be represented as:

- the position of the "primary" vertex 'p0' plus...
- a proportional distance 'u' along a 1st edge plus...
- a proportional distance 'v' along a 2nd edge

```
r = p0 + u(p1-p0) + v(p2-p0)
```

You must surely remember

Barycentric coordinates

From way back in WB2 ?

(and the in-class test !)

Ray/Triangle Intersection Code

We can rearrange formulae to find intersection point

We won't go through full derivation of this right now

(see narrated video in workbook if you're interested)

Instead we'll just give you the required C++ code

Note that the "solution" vec3 is [t,u,v] NOT [x,y,z]

```
glm::vec3 e0 = triangle.vertices[1] - triangle.vertices[0];
glm::vec3 e1 = triangle.vertices[2] - triangle.vertices[0];
glm::vec3 SPVector = cameraPosition - triangle.vertices[0];
glm::mat3 DEMatrix(-rayDirection, e0, e1);
glm::vec3 possibleSolution = glm::inverse(DEMatrix) * SPVector;
```

Validating Intersections

Formulae give intersection between ray and PLANE
We really only want a REGION on a plane (triangle)

Just as we did with the triangular colour spectrum
We must validate `u` and `v` to make sure that they are both within the BOUNDS of the triangle:

```
0 <= u <= 1
0 <= v <= 1
(u + v) <= 1
```

UnconstrainedBarycentric

Additional Validation Checks

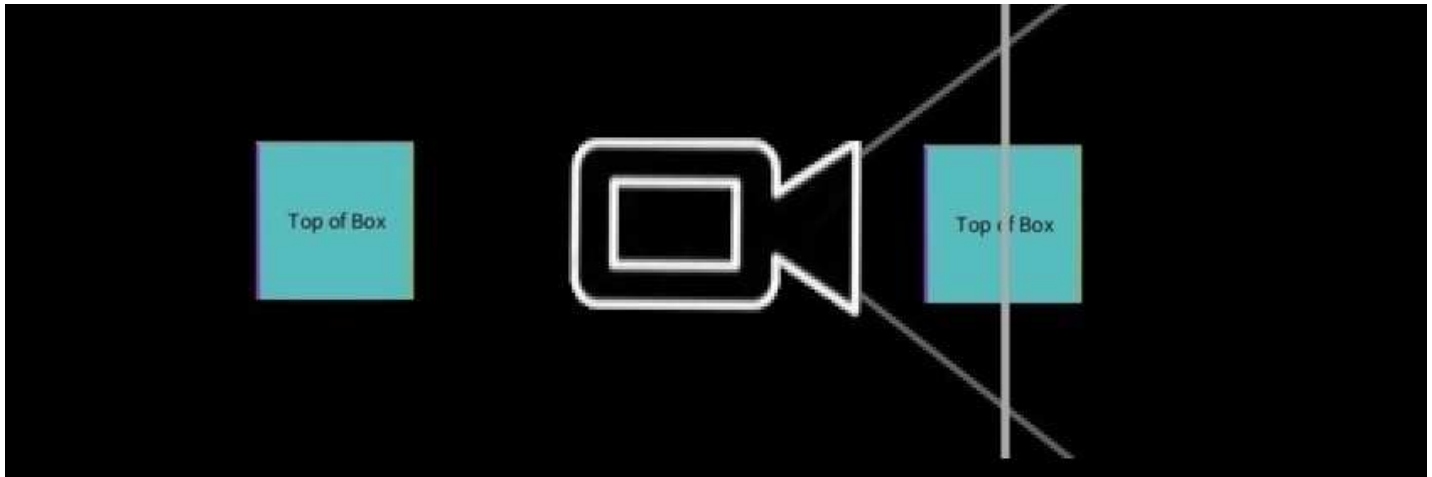
Now that we are working in 3D (rather than just 2D)

We must ALSO consider the Z-depth of an intersection

Obviously intersections *behind* camera are "bad"

But we can go one step further and add a refinement:

Ignore any intersection *closer* than the image plane



Converting [t,u,v] to a Position in 3D

Once we have a valid [t,u,v] we need to convert it
In order to derive a true position in 3D world space

We have two formulae, so we can either...

- Insert 't' into the point-along-a-ray formula:

```
r = s + (t * d)
```

- Or insert 'u' and 'v' into the plane/triangle formula:

```
r = p0 + u(p1-p0) + v(p2-p0)
```

Both approaches *should* give same intersection
Why not calculate both - by way of a double-check ?

Warning about Return Values

What will your 'getClosestValidIntesection' return ?
Typically people return a 'ModelTriangle' instance
(i.e. the closest triangle that has been intersected)

But what if the ray was fired into "empty space" ?
There is no intersection, so what value is returned ?

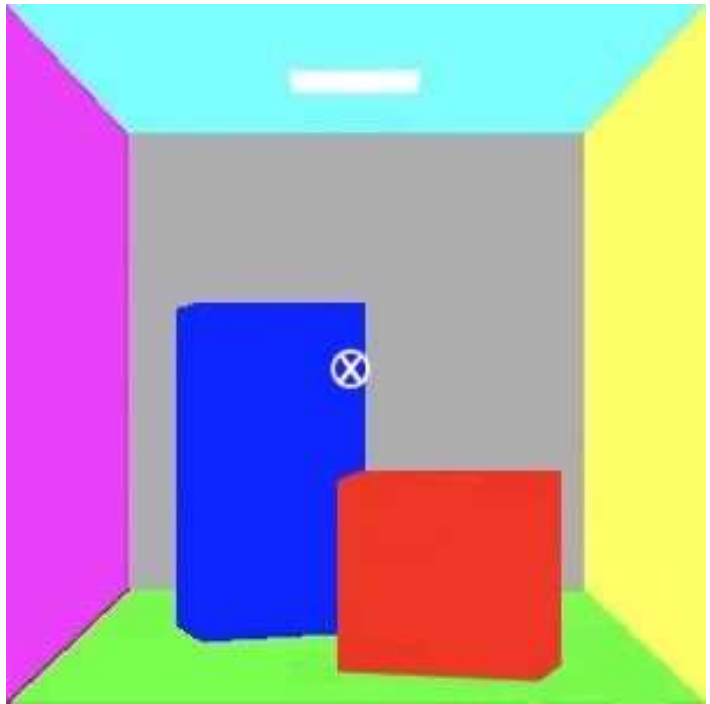
Danger that code interprets "no result" as a triangle
Can lead to some strange effects (especially in C++)

Testing your Closest Intersection Function

Try firing a ray straight into the middle of the scene

It *should* intersect with the blue box and back wall

A triangle belonging to the blue box should be closest



But Why ?

Up until this point, it seems that "Ray Tracing"
Is just a HARDER way (more processor-intensive)
to achieve the SAME results as Rasterising !

There are however some positive benefits...

Maintaining Z-depth

Due to the fact that edges 'e0' and 'e1' are 3D
Our position 'r' on the triangle is ALSO in 3D space

When we flattened triangles for rasterising in WB4
We LOST Z-depth of vertices (and surface points)

We later had to reconstruct z-depths (for texturing)
When we had to deal with challenge of perspective

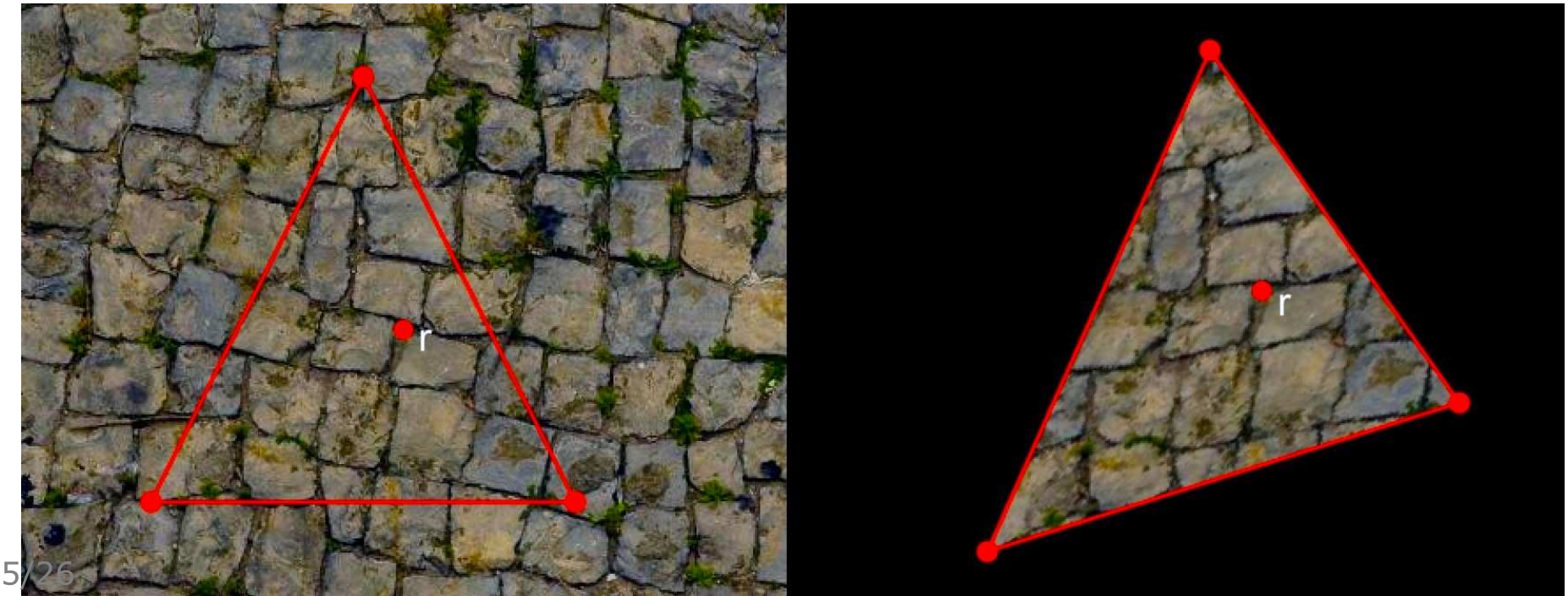
But with Ray Tracing, we KEEP Z-depth at all times
Makes texture mapping a LOT easier (even trivial ?)

Ray-Traced Texture Mapping

u, v, w are SAME for BOTH texture AND 3D triangle

Each 2D vertex on texture map has [x,y] coords

Calculate a weighted average for texture point [x,y]

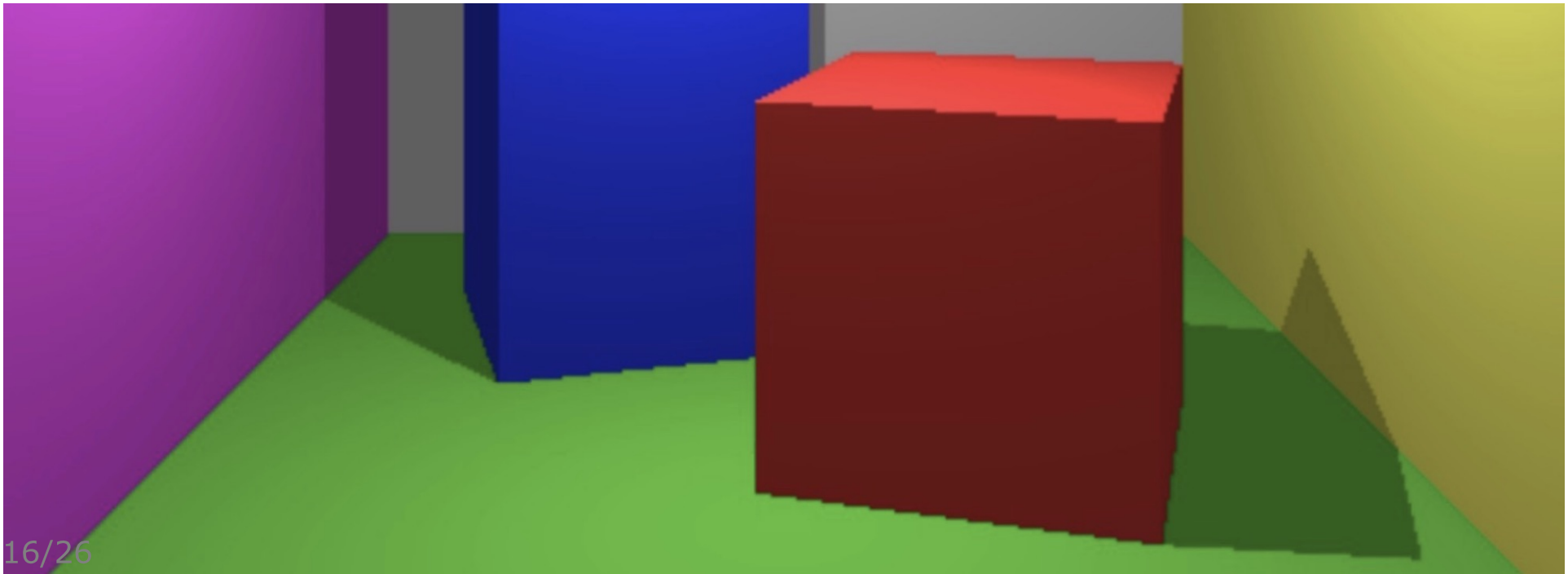


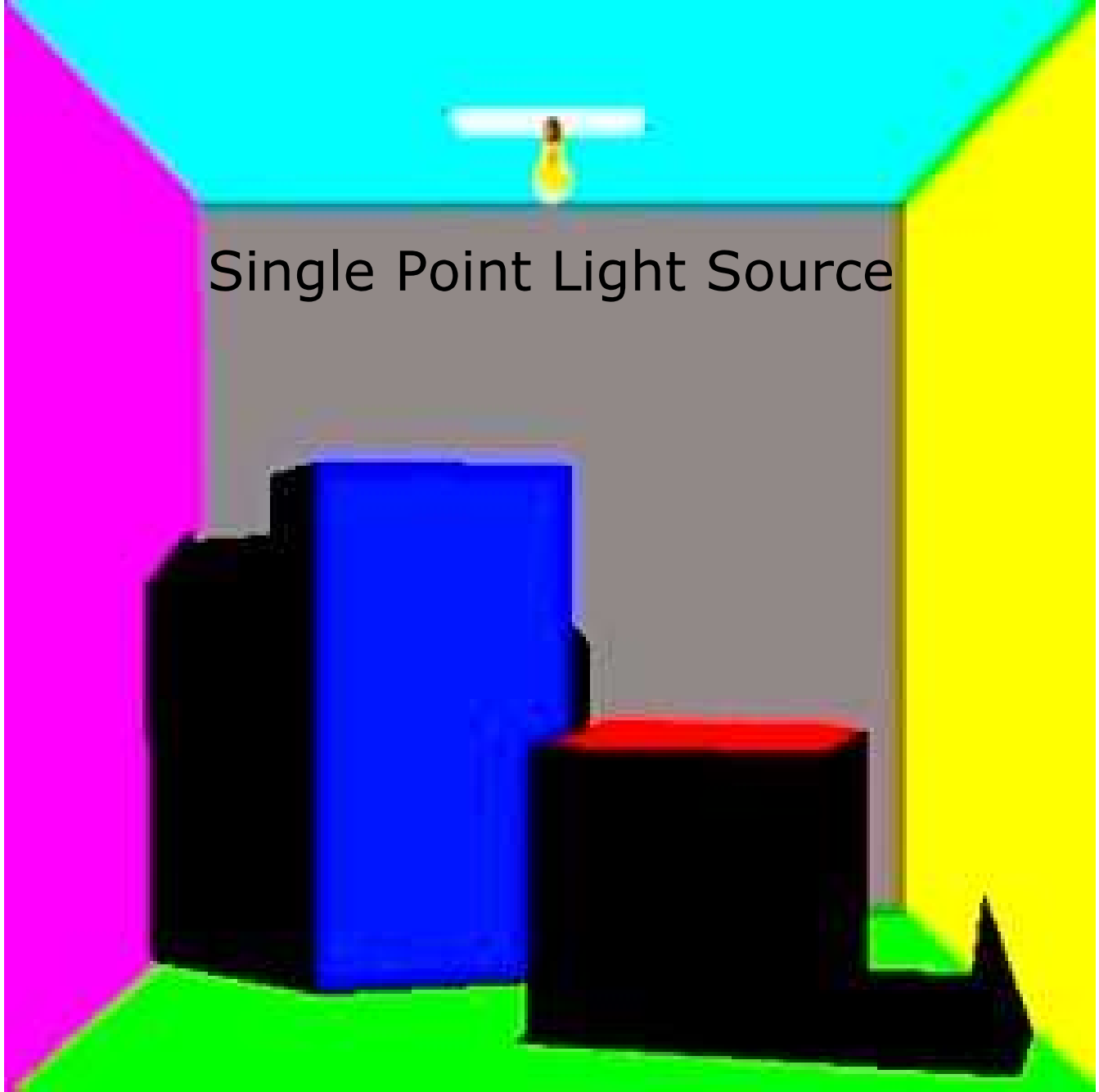
New lighting effects are possible !

Ray Tracing permits various lighting effects to be used

For example, creating Shadows is conceptually easy...

"Can a particular point on a surface *SEE* the light ?"




Single Point Light Source

Reuse of Existing Function

We already have function to find closest intersection
Could we reuse this function to help with shadows ?

If we fire a ray from the "possibly in shadow" point
For "direction" use the vector towards light source

If the intersection is beyond light: fully illuminated
If intersection is between surface and light: shadow
If there is no intersection at all: fully illuminated

May need to refactor function a little for it to work !

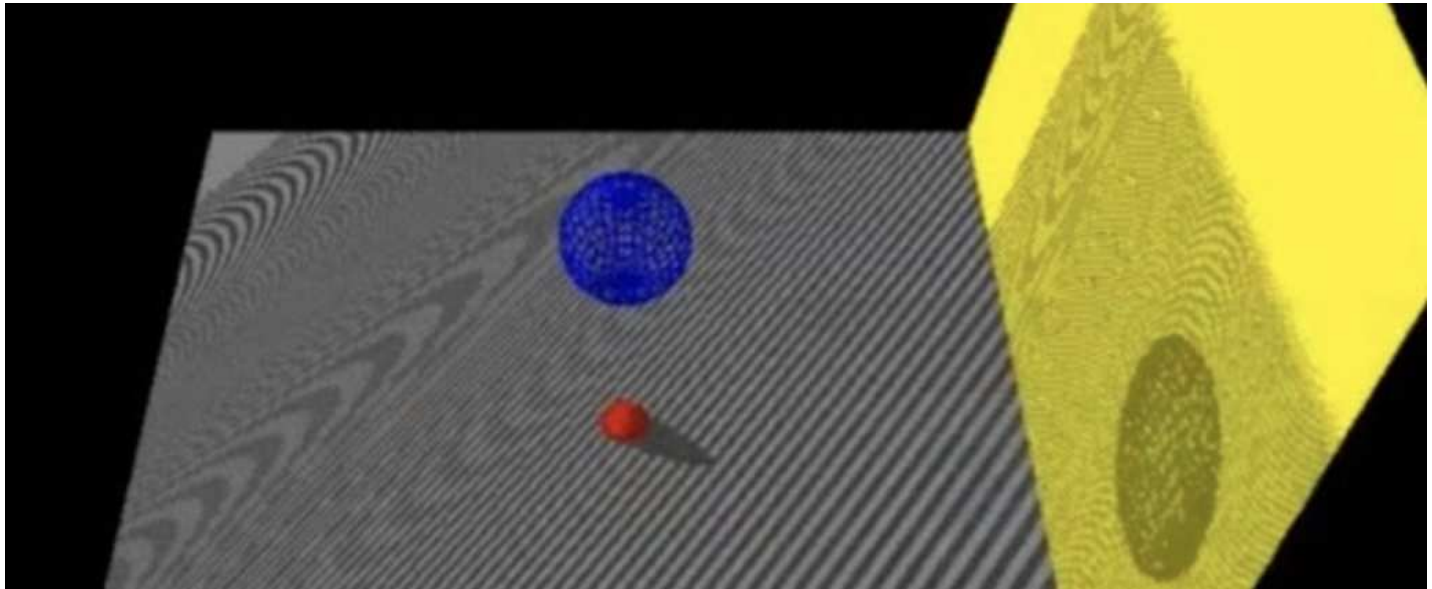
Shadow Acne

You may encounter phenomenon of "Shadow Acne"

Shadow pixels are drawn where they should NOT be

Caused by shadow ray "seeing" originating surface

ShadowAcne





Possible Solutions

Check the index of the triangle casting the shadow
It's NOT a shadow IF triangle causing the shadow...
Is the SAME as the triangle being overshadowed !

Alternatively implement an "exclusion zone" policy
A triangle can't be overshadowed by ANY triangle
That is VERY close to it (which includes itself)
This might lead to some "false negative" results
But it shouldn't have a big impact on your render

Keep your Rasterising Code !

This isn't to say that Rasterising was wasted time
You'll still need Rasterising code you have written

The speed of rasterising is one of its main benefits

It will be useful to be able to switch between modes
It is also possible for us to render "hybrid" scenes
(Rasterising some elements, Ray Tracing others)

A Few Thoughts About Debugging

Debugging Tips

As you are probably starting to realise...
Debugging 3D renders can be particularly tricky !
Problem is the large amount of data washing around
You can't just print everything out !

IDE debuggers can be very helpful
But sometimes you need additional strategies...
A useful approach is "selective" rendering:

SingleTriangleRenderer

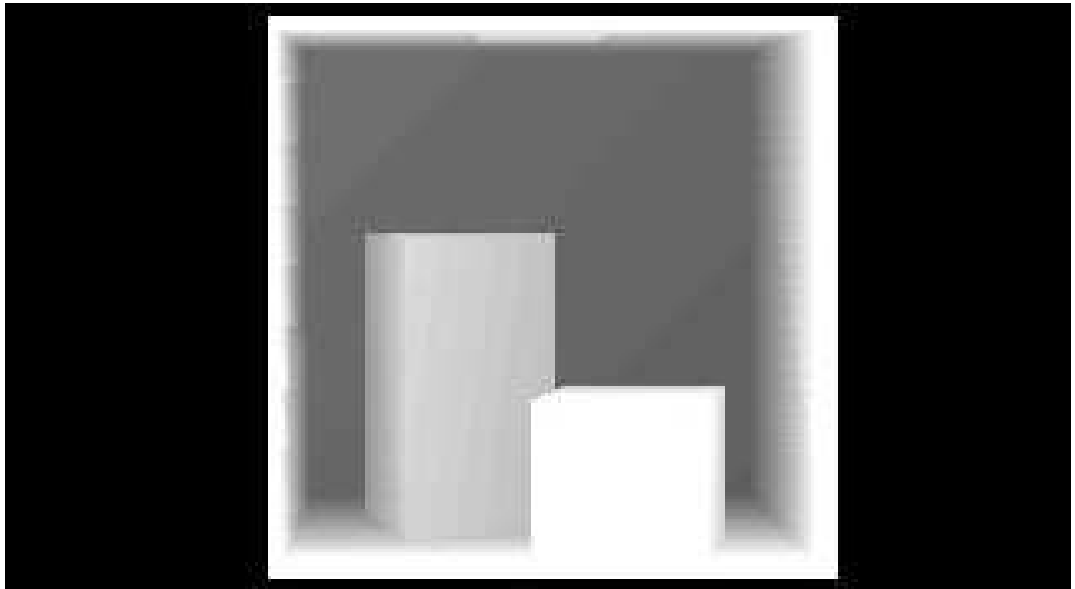
"Visual" Debugging

Rather than drawing pixels using their "true" colour

We can colour them according to a numerical value

For example, the distance of points from the camera

Easier to see patterns than with raw numerical data



Follow me for more tips !

We've put together a document of debugging tips

README

Have put it in the "Debugging" folder on GitHub !

